Université des Sciences et de la Technologie Houari Boumediene Faculté d'Informatique — Module : ATBD	M1 SSI — 2024-2025 Examen Final — Durée : 1h30 min

CORRIGÉ TYPE — EXAMEN ATBD 2024-2025

EXERCICE 1 : Sécurité Oracle

Question 1. Quelle combinaison décrit le plus efficacement une politique de sécurité Oracle robuste pour restreindre l'accès en lecture seule à une table PAIE contenant des données sensibles ?

✔ **Réponse correcte :**

Attribuer uniquement les privilèges SELECT sur la table PAIE et créer une vue en lecture seule.

> ***Justification :*** *Accorder uniquement SELECT garantit qu'aucune modification n'est possible. La vue en lecture seule masque la structure réelle. Les autres options accordent trop de droits (INSERT, UPDATE, ALL PRIVILEGES).*

Question 2. Écrivez les commandes SQL pour créer le profil puis l'utilisateur « auditeur_financier » (session 30 min, verrouillage après 3 tentatives, mdp tous les 45 jours, non réutilisable pendant 180 jours).

✔ **Réponse correcte :**

```
CREATE PROFILE Profil_auditeur LIMIT
  CONNECT_TIME          30
  FAILED_LOGIN_ATTEMPTS 3
  PASSWORD_LIFE_TIME    45
  PASSWORD_REUSE_TIME   180;

CREATE USER auditeur_financier
  IDENTIFIED BY MonMotDePasse1
  PROFILE Profil_auditeur;
```

Question 3. Politique d'audit qui détecte toute tentative de DELETE sur EMPLOYES, même si elle échoue.

✔ **Réponse correcte :**

```
-- Syntaxe unifiée (capture succès ET échecs) :
AUDIT DELETE ON EMPLOYES;

-- Ou pour cibler uniquement les échecs :
AUDIT DELETE ON EMPLOYES WHENEVER NOT SUCCESSFUL;
```

> ***Justification :*** *WHENEVER NOT SUCCESSFUL capture les tentatives échouées. WHENEVER SUCCESSFUL ne capte que les suppressions réussies, ce qui ne répond pas à l'exigence. Sans clause WHENEVER, Oracle enregistre les deux cas.*

Question 4. FINANCE_USER doit consulter SALAIRES uniquement pendant les heures ouvrables (8h–18h). Quelle technologie Oracle est la plus adaptée ?

✔ **Réponse correcte :**

C. VPD (Virtual Private Database) avec une politique conditionnelle.

Justification : VPD permet d'ajouter dynamiquement une clause WHERE selon des conditions contextuelles comme l'heure (SYS_CONTEXT). Les profils avec IDLE_TIME gèrent l'inactivité, pas les plages horaires.

Question 5. Combinaison de paramètres de profil la plus efficace contre les attaques par force brute sur les mots de passe ?

✔ **Réponse correcte :**

B. FAILED_LOGIN_ATTEMPTS + PASSWORD_LOCK_TIME

Justification : FAILED_LOGIN_ATTEMPTS fixe le nombre max de tentatives échouées avant verrouillage. PASSWORD_LOCK_TIME définit la durée du verrouillage. Ensemble ils bloquent directement les attaques par force brute.

Question 6. Un administrateur veut que les privilèges accordés indirectement soient aussi révoqués lors de la suppression d'un privilège. Quelle clause SQL doit-il utiliser ?

✔ **Réponse correcte :**

B. REVOKE … FROM … CASCADE

Justification : CASCADE propage la révocation aux utilisateurs qui ont reçu le privilège via GRANT OPTION. Sans CASCADE, les privilèges accordés à d'autres restent actifs, créant une faille de sécurité.

Question 7. Objectif principal de PASSWORD_VERIFY_FUNCTION dans un profil Oracle ?

✔ **Réponse correcte :**

Imposer des règles de complexité sur les mots de passe. La fonction PL/SQL spécifiée est appelée automatiquement lors de chaque création ou modification de mot de passe. Elle peut vérifier la longueur minimale, la présence de chiffres, de majuscules, de caractères spéciaux, etc.

Question 8. Lequel des éléments suivants N'EST PAS une contre-mesure directement gérée par le SGBD Oracle ?

✔ **Réponse correcte :**

B. Restrictions réseau (coupe-circuit / firewall)

Justification : Les restrictions réseau sont gérées au niveau système/réseau, pas par Oracle. Les rôles, profils et limites de ressources sont des mécanismes internes au SGBD.

Question 9. Quel mécanisme Oracle permet d'appliquer automatiquement des filtres de sécurité personnalisés au niveau ligne pour chaque utilisateur ?

✔ **Réponse correcte :**

B. Virtual Private Database (VPD)

Justification : VPD (aussi appelé Row Level Security) attache une fonction PL/SQL à une table. Cette fonction génère automatiquement une clause WHERE ajoutée à chaque requête, de façon transparente pour l'utilisateur.

Question 10. Commande SQL pour auditer toute tentative DELETE sur RH.SALAIRES, réussie ou non.

✔ **Réponse correcte :**

```
-- Capture les deux cas (succès + échec) :
AUDIT DELETE ON RH.SALAIRES;
```

```
-- Ou avec politique nommée :
CREATE AUDIT POLICY audit_salaires
  ACTIONS DELETE ON RH.SALAIRES;
AUDIT POLICY audit_salaires;
```

Question 11. Commande pour afficher les accès de userx à la table CLIENTS en dehors des heures de bureau à partir du journal d'audit Oracle.

✔ **Réponse correcte :**

```
SELECT *
FROM DBA_AUDIT_TRAIL
WHERE DB_USER = 'USERX'
  AND OBJ_NAME = 'CLIENTS'
  AND (TO_NUMBER(TO_CHAR(TIMESTAMP,'HH24')) < 8
       OR TO_NUMBER(TO_CHAR(TIMESTAMP,'HH24')) >= 18);
```

Question 12. Combinaison garantissant que les utilisateurs n'accèdent qu'aux enregistrements correspondant à leur service ?

✔ **Réponse correcte :**

C. Politique VPD avec fonction PL/SQL attachée à la table.

Justification : *C'est le cas d'usage typique de VPD : filtrer automatiquement les lignes visibles par chaque utilisateur selon son contexte (service, département, etc.).*

Question 13. Commande SQL permettant à l'administrateur d'accorder à manager_rh le droit de créer des utilisateurs ET de déléguer ce droit à d'autres.

✔ **Réponse correcte :**

```
GRANT CREATE USER TO manager_rh WITH ADMIN OPTION;
```

Justification : *WITH ADMIN OPTION permet à manager_rh de ré-accorder le privilège système CREATE USER à d'autres utilisateurs. C'est l'équivalent de WITH GRANT OPTION pour les privilèges système.*

Question 14. Que se passe-t-il si la limite CPU_PER_SESSION 5000 est dépassée pendant l'exécution d'une session ?

✔ **Réponse correcte :**

Oracle met fin à la session de l'utilisateur et génère l'erreur ORA-02392. La session est déconnectée de force. L'utilisateur doit se reconnecter pour continuer.

Question 15. Quel est l'intérêt de l'option CASCADE dans une commande REVOKE ?

✔ **Réponse correcte :**

B. Elle empêche les utilisateurs indirectement autorisés de conserver le privilège.

Justification : *Si A a accordé un privilège à B avec GRANT OPTION, et B l'a accordé à C, révoquer le privilège de B avec CASCADE révoque automatiquement aussi celui de C. Sans CASCADE, C conserverait le privilège (faille de sécurité).*

Question 16. Auditer chaque tentative de lecture ou modification RÉUSSIE sur DONNEES_CONFIDENTIELLES.

✔ **Réponse correcte :**

```
AUDIT SELECT, INSERT, UPDATE
  ON DONNEES_CONFIDENTIELLES
  WHENEVER SUCCESSFUL;
```

Question 17. Le département RH souhaite que chaque utilisateur RH ne voie que les lignes où DEPARTEMENT = 'RH' dans la table EMPLOYES. Quelle solution Oracle est la plus appropriée ?

✔ **Réponse correcte :**

C. Utiliser une politique Oracle VPD avec une fonction filtrant par département.

Code SQL :

```
-- Étape 1 : créer la fonction de filtre
CREATE OR REPLACE FUNCTION filtre_rh (
  p_schema IN VARCHAR2,
  p_object IN VARCHAR2
) RETURN VARCHAR2 AS
BEGIN
  RETURN 'DEPARTEMENT = ''RH''';
END;
/

-- Étape 2 : attacher la politique VPD à la table
BEGIN
  DBMS_RLS.ADD_POLICY(
    object_schema   => 'RH',
    object_name     => 'EMPLOYES',
    policy_name     => 'politique_filtre_RH',
    function_schema => 'RH',
    policy_function => 'FILTRE_RH'
  );
END;
/
```

Question 18. Bloquer automatiquement un compte après 5 tentatives de connexion échouées, pour une durée de 1 jour.

✔ **Réponse correcte :**

```
CREATE PROFILE profil_securise LIMIT
  FAILED_LOGIN_ATTEMPTS 5
  PASSWORD_LOCK_TIME    1;  -- 1 = 1 jour

ALTER USER nom_utilisateur PROFILE profil_securise;
```

Question 19. Meilleure politique pour surveiller les connexions des utilisateurs ayant des privilèges DBA.

✔ **Réponse correcte :**

```
AUDIT SESSION WHENEVER SUCCESSFUL;
AUDIT SESSION WHENEVER NOT SUCCESSFUL;
-- Ou pour cibler spécifiquement les DBA :
AUDIT SESSION BY DBA;
```

Question 20. Accorder temporairement un accès complet à la table FACTURATION à CONSULTANT_X, de façon facilement révocable. Quelle est la meilleure approche ?

✔ **Réponse correcte :**

A. Créer un rôle conseil, lui attribuer les droits, et attribuer le rôle à l'utilisateur.

```
-- Créer le rôle
CREATE ROLE role_consultant;
```

```
-- Accorder les droits au rôle
GRANT ALL ON FACTURATION TO role_consultant;

-- Attribuer le rôle à l'utilisateur
GRANT role_consultant TO CONSULTANT_X;

-- Révoquer facilement plus tard
REVOKE role_consultant FROM CONSULTANT_X;
```

Justification : *Passer par un rôle est la meilleure pratique : révocation en une seule commande, rôle réutilisable, gestion centralisée des droits.*

EXERCICE 2 : Optimisation et Plans d'Exécution Oracle

Schéma de la base (compagnie d'assurance automobile) :

```
ASSURE   (NOASSURE, NOM-ASS, ADRESSE-ASS, AGE-ASS, WILAYA-ASS)
VEHICULE (NOVEHICULE, TYPE, PUISSANCE, MARQUE, ANNEE, WILAYA-VEH)
CONTRAT  (NOCONTRAT, NOASSURE*, NOVEHICULE*, DATE-C, NATURE, PRIME)
SINISTRE (NOSINISTRE, NOCONTRAT*, DATE-S, CAUSE)
-- * = clé étrangère    _ = clé primaire soulignée
```

Requête R1 analysée :

```
SELECT S.NOSINISTRE, S.DATE_S, S.CAUSE
FROM   SINISTRE S, CONTRAT C, VEHICULE V
WHERE  V.MARQUE       = 'FIAT'
  AND  C.NATURE       = 'totale'
  AND  S.NOCONTRAT    = C.NOCONTRAT
  AND  C.NOVEHICULE   = V.NOVEHICULE;
```

Plan d'exécution initial :

ID	Opération	Nom	Rows	Cost	Access Path
0	SELECT STATEMENT			45	
1	NESTED LOOPS		10	45	
2	NESTED LOOPS		10	30	
3	TABLE ACCESS BY INDEX ROWID	VEHICULE	20	10	INDEX RANGE SCAN (MARQUE_IDX)
4	TABLE ACCESS BY INDEX ROWID	CONTRAT	10	15	INDEX RANGE SCAN (NOVEHICULE_IDX)
5	TABLE ACCESS BY INDEX ROWID	SINISTRE	10	15	INDEX RANGE SCAN (NOCONTRAT_IDX)

Question 1. Quels types de scans sont utilisés ? Expliquer.

✔ Réponse correcte :

Le plan utilise exclusivement des INDEX RANGE SCAN sur trois index différents. Un INDEX RANGE SCAN est utilisé lorsqu'Oracle peut délimiter une plage de valeurs dans l'index grâce à un prédicat (égalité ou inégalité). Plutôt que de lire toute la table (Full Table Scan), Oracle parcourt uniquement la portion de l'index correspondant au prédicat, puis accède directement aux lignes via leur ROWID. C'est beaucoup plus rapide quand peu de lignes sont concernées.

Les trois scans correspondent à :

- MARQUE_IDX → filtre V.MARQUE = 'FIAT'
- NOVEHICULE_IDX → jointure C.NOVEHICULE
- NOCONTRAT_IDX → jointure S.NOCONTRAT

Question 2. Quels index sont exploités dans ce plan ? Donner le code de création.

✔ Réponse correcte :

```
-- Index sur VEHICULE pour le filtre MARQUE
CREATE INDEX MARQUE_IDX ON VEHICULE(MARQUE);

-- Index sur CONTRAT pour la jointure NOVEHICULE
```

```
CREATE INDEX NOVEHICULE_IDX ON CONTRAT(NOVEHICULE);

-- Index sur SINISTRE pour la jointure NOCONTRAT
CREATE INDEX NOCONTRAT_IDX ON SINISTRE(NOCONTRAT);
```

Question 3. Quelle est la méthode de jointure utilisée ? Justifier le choix.

✔ **Réponse correcte :**

La méthode utilisée est le Nested Loops Join (jointure par boucles imbriquées).

Fonctionnement : Pour chaque ligne de la table externe (VEHICULE filtrée sur MARQUE = 'FIAT'), Oracle cherche les lignes correspondantes dans la table interne (CONTRAT) via l'index NOVEHICULE_IDX, puis répète l'opération pour SINISTRE via NOCONTRAT_IDX.

> ***Justification :*** *Cette méthode est optimale ici car les filtres sont très sélectifs, le nombre de lignes intermédiaires est faible (10 à 20 lignes), et les index permettent un accès direct sans lire les tables entières.*

Question 4. Le plan est-il optimal en termes de performances ?

✔ **Réponse correcte :**

Le plan initial est globalement bon mais pas nécessairement optimal dans tous les contextes.

Points positifs : index bien exploités, coût de 45, Nested Loops adaptés à la faible cardinalité des résultats intermédiaires.

Limite potentielle : si VEHICULE contient beaucoup de véhicules FIAT (faible sélectivité de MARQUE_IDX), Oracle ferait de nombreuses itérations dans la boucle externe, dégradant les performances. Le plan alternatif avec Hash Join pourrait alors être plus efficace.

Question 5. Y a-t-il des améliorations possibles, comme la création d'index supplémentaires ?

✔ **Réponse correcte :**

Oui, deux améliorations principales sont envisageables :

1. Créer un index composite sur CONTRAT(NOVEHICULE, NATURE) pour couvrir à la fois la jointure et le filtre sur NATURE en un seul accès index :

```
CREATE INDEX CONTRAT_VEH_NAT_IDX ON CONTRAT(NOVEHICULE, NATURE);
```

2. Mettre à jour les statistiques pour que l'optimiseur évalue correctement les cardinalités :

```
EXEC DBMS_STATS.GATHER_TABLE_STATS('RH','VEHICULE');
EXEC DBMS_STATS.GATHER_TABLE_STATS('RH','CONTRAT');
EXEC DBMS_STATS.GATHER_TABLE_STATS('RH','SINISTRE');
```

Plan d'exécution alternatif (après optimisation) :

0	SELECT STATEMENT			30
1	HASH JOIN		10	30
2	HASH JOIN		20	20
3	TABLE ACCESS FULL	VEHICULE	800	10
4	TABLE ACCESS FULL	CONTRAT	1600	10
5	TABLE ACCESS FULL	SINISTRE	256	10

Question 6. Interpréter le nouveau plan d'exécution.

✔ Réponse correcte :

Le nouveau plan utilise des Hash Joins avec des Full Table Scans sur les trois tables. Oracle lit complètement VEHICULE (800 lignes), CONTRAT (1600 lignes) et SINISTRE (256 lignes). Pour chaque Hash Join, Oracle charge la table la plus petite en mémoire dans une table de hachage, puis parcourt la plus grande en cherchant les correspondances par hachage. Les deux Hash Joins sont imbriqués : d'abord VEHICULE + CONTRAT, puis le résultat + SINISTRE. Le coût total est réduit à 30 (contre 45 dans le plan initial).

Question 7. Quels sont les avantages et inconvénients de chaque plan ?

✔ Réponse correcte :

Plan initial — Nested Loops + Index Range Scan :

> Avantages : Très efficace pour des petits volumes ; accès précis via ROWID sans lire toute la table ; faible consommation mémoire ; idéal en OLTP.
>
> Inconvénients : Devient lent si la table externe retourne beaucoup de lignes ; dépend de la qualité des index ; coût estimé plus élevé (45).

Plan alternatif — Hash Join + Full Table Scan :

> Avantages : Coût estimé plus bas (30) ; efficace pour les grandes tables et jointures volumineuses ; ne dépend pas des index.
>
> Inconvénients : Consomme beaucoup de mémoire pour les tables de hachage ; lit toutes les lignes même celles non pertinentes ; moins adapté aux requêtes transactionnelles.

Question 8. Dans quel contexte l'un serait-il préférable à l'autre ?

✔ Réponse correcte :

Nested Loops + Index : préférable en contexte OLTP (Online Transaction Processing) — requêtes fréquentes et ciblées, peu de lignes retournées, temps de réponse critique.

Hash Join + Full Scan : préférable en contexte décisionnel / Data Warehouse — requêtes analytiques sur de grands volumes, batchs nocturnes, rapports, où la mémoire disponible est suffisante.

Question 9. Comment la présence ou l'absence d'index influence-t-elle le choix du plan par l'optimiseur ?

✔ Réponse correcte :

L'optimiseur Oracle (CBO — Cost Based Optimizer) évalue le coût de plusieurs stratégies et choisit la moins coûteuse.

Avec des index sur les colonnes de filtre et de jointure : l'optimiseur calcule le coût d'un accès par index (Index Range Scan + ROWID), très faible si la sélectivité est bonne. Il préférera alors les Nested Loops.

Sans index : l'optimiseur est contraint aux Full Table Scans et compare Hash Join vs Sort Merge Join pour choisir le moins coûteux.

Si un index existe mais est peu sélectif (nombreuses lignes retournées), l'optimiseur peut décider de l'ignorer et de faire un Full Table Scan, car les nombreux accès ROWID individuels (I/O aléatoires) peuvent être plus coûteux que la lecture séquentielle.

Question 10. Expliquez le rôle de la sélectivité des colonnes dans la décision de l'optimiseur.

✔ Réponse correcte :

La sélectivité d'une colonne mesure la proportion de lignes retournées par un prédicat. Elle est calculée à partir des statistiques : sélectivité = 1 / NDV (NDV = Number of Distinct Values).

Colonne très sélective (NDV élevé, peu de lignes retournées) : l'index est très utile. Par exemple, si MARQUE = 'FIAT' ne représente que 2% des véhicules, l'index MARQUE_IDX évite de lire 98% de la table.

Colonne peu sélective (NDV faible, beaucoup de lignes retournées) : l'index devient peu intéressant. Oracle préférera un Full Table Scan + Hash Join, car la lecture séquentielle est plus rapide que des milliers d'accès ROWID aléatoires.

Conclusion : plus un prédicat est sélectif, plus un index sur cette colonne est bénéfique et plus l'optimiseur sera enclin à choisir un plan avec Nested Loops + Index Scan.